

Arrays & strings

COMPLETE LEARNING GUIDE

What You'll Learn

Array and string problems dominate the early rounds of technical interviews because they require no special data structure knowledge to get started, yet they reward genuinely different algorithmic thinking than the brute-force nested loop most candidates reach for first. This submodule focuses purely on algorithmic technique — unlike the Java Fundamentals "Arrays & strings" submodule, which covered Java's array/String syntax and mechanics, this one assumes that mechanics and teaches the patterns that turn an $O(n^2)$ brute-force scan into an $O(n)$ or $O(n \log n)$ solution: the two-pointer technique, the sliding window technique, prefix sums, and in-place array manipulation. Each pattern is demonstrated with a full worked Java example and an explicit statement of its time and space complexity, because stating complexity correctly and unprompted is exactly what separates a strong interview answer from an average one.

1. The Two-Pointer Technique

Two-pointer problems maintain two indices into the same array (or string) that move toward each other or in the same direction based on a comparison, replacing what would otherwise be an $O(n^2)$ pair-checking loop with a single $O(n)$ pass. It's the standard approach for problems on a sorted array where you need to find a pair, triple, or subrange satisfying some condition — the classic example is finding two numbers in a sorted array that sum to a target: start one pointer at each end, and move the low pointer up if the current sum is too small, or the high pointer down if it's too large, since moving in the "wrong" direction could never help.

JAVA

```
// Two Sum on a SORTED array -- two-pointer, O(n) time, O(1) space
public static int[] twoSumSorted(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left < right) {
        int sum = arr[left] + arr[right];
        if (sum == target) return new int[]{left, right};
        if (sum < target) left++;    // need a bigger sum -- move low pointer up
        else right--;               // need a smaller sum -- move high pointer
    }
    return new int[]{-1, -1};      // no pair found
}
// Time: O(n) -- each pointer moves at most n times total
// Space: O(1) -- no extra data structure
```

2. Sliding Window: Fixed and Variable Size

Sliding window problems maintain a contiguous subrange (the "window") over an array or string and slide it forward one element at a time, updating a running aggregate incrementally instead of recomputing it from scratch for every possible window — this turns an $O(n \cdot k)$ brute force (recomputing each window of size k from zero) into $O(n)$. A fixed-size window (find the maximum sum of any k consecutive elements) adds the new element entering the window and subtracts the one leaving it on each step. A variable-size window (find the smallest subarray with sum $\geq$ target) expands the right edge until a condition is met, then contracts the left edge while the condition still holds, tracking the best window seen.

JAVA

```
// Maximum sum of any window of size k -- fixed-size sliding window,  $O(n)$ 
public static int maxSumWindow(int[] arr, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += arr[i];    // sum the first window
    int maxSum = windowSum;

    for (int i = k; i < arr.length; i++) {
        windowSum += arr[i] - arr[i - k];    // add new element, remove the one
        // leaving the window
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
// Time:  $O(n)$  -- each element is added and removed from the window exactly once
// Space:  $O(1)$ 
```

3. Prefix Sums for Fast Range Queries

A prefix sum array precomputes, for each index i , the sum of all elements from the start of the array up to i — once built (a single $O(n)$ pass), the sum of any arbitrary range $[i, j]$ can be answered in $O(1)$ as $\text{prefix}[j] - \text{prefix}[i-1]$, instead of re-summing that range in $O(n)$ every time it's queried. This pattern is essential whenever a problem asks for many range-sum queries over the same static array — the naive approach of summing each range on demand is $O(n)$ per query and $O(n \cdot q)$ overall for q queries, while prefix sums reduce total query time to $O(q)$ after one $O(n)$ preprocessing pass.

JAVA

```
// Build once:  $O(n)$ . Answer each range-sum query in  $O(1)$  after that.
public static int[] buildPrefixSums(int[] arr) {
    int[] prefix = new int[arr.length + 1];    // prefix[0] = 0 by convention
    for (int i = 0; i < arr.length; i++) {
        prefix[i + 1] = prefix[i] + arr[i];
    }
    return prefix;
}

public static int rangeSum(int[] prefix, int left, int right) {    // inclusive
    // [left, right]
    return prefix[right + 1] - prefix[left];
}
```

4. In-Place Array Manipulation

Many array problems ask for a transformation "in-place," meaning without allocating a second array proportional to the input size, which forces $O(1)$ extra space rather than the $O(n)$ an obvious second-array approach would use. Common in-place techniques include reversing a subrange by swapping from both ends inward, partitioning elements around a pivot (as in quicksort's partition step or the Dutch national flag problem), and the "write pointer" pattern for removing elements in-place — a slow pointer tracks where the next kept element should be written, while a fast pointer scans the whole array, so unwanted elements are simply never copied forward.

JAVA

```
// Remove all occurrences of 'target' in-place, return new logical length
public static int removeInPlace(int[] arr, int target) {
    int writeIndex = 0; // slow pointer: next position to write a kept element
    for (int readIndex = 0; readIndex < arr.length; readIndex++) { // fast pointer: scans everything
        if (arr[readIndex] != target) {
            arr[writeIndex] = arr[readIndex];
            writeIndex++;
        }
    }
    return writeIndex; // elements [0, writeIndex) are the result
}
// Time:  $O(n)$ , Space:  $O(1)$  -- no second array allocated
```

5. String-Specific Techniques: Palindromes and Anagrams

Palindrome checks are a direct application of the two-pointer technique on a String or char[]: compare characters from both ends moving inward, and any mismatch proves it's not a palindrome, short-circuiting immediately rather than always doing a full pass. Anagram checks (do two strings contain exactly the same characters, possibly reordered?) are most efficiently solved with a frequency-count array or HashMap rather than sorting both strings — sorting costs $O(n \log n)$, while counting character frequencies costs $O(n)$, a meaningful complexity improvement interviewers specifically look for.

JAVA

```
// Anagram check via frequency counting --  $O(n)$  instead of  $O(n \log n)$  via sorting
public static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] counts = new int[26]; // assumes lowercase a-z
    for (char c : a.toCharArray()) counts[c - 'a']++;
    for (char c : b.toCharArray()) counts[c - 'a']--;
    for (int count : counts) {
        if (count != 0) return false;
    }
    return true;
}
// Time:  $O(n)$ , Space:  $O(1)$  -- the counts array is fixed size (26), independent of input length
```

Common Pitfalls

- Defaulting to nested loops ($O(n^2)$) for pair-sum or subarray problems without first checking whether two-pointer or sliding window reduces it to $O(n)$.
- Applying two-pointer to an unsorted array expecting sorted-array behavior — two-pointer's directional logic only works correctly once the array is sorted (or the problem doesn't require order).
- Recomputing a sliding window's sum from scratch on every shift instead of incrementally adding the entering element and subtracting the leaving one.
- Off-by-one errors in prefix sum ranges — forgetting $\text{prefix}[0] = 0$ as a sentinel, or mixing up inclusive vs. exclusive range boundaries.
- Allocating a second array for an "in-place" transformation, missing the $O(1)$ extra-space requirement the problem is actually testing.
- Sorting both strings to check for an anagram ($O(n \log n)$) instead of frequency counting ($O(n)$), missing an easy complexity improvement.
- Not stating time and space complexity out loud after presenting a solution — interviewers expect this proactively, not only when asked.

Key Takeaways

- Two-pointer converts many $O(n^2)$ pair/range problems on sorted data into $O(n)$ by moving two indices based on a comparison.
- Sliding window converts $O(n \cdot k)$ windowed-aggregate problems into $O(n)$ by updating the aggregate incrementally instead of recomputing it.
- Prefix sums answer arbitrary range-sum queries in $O(1)$ each after one $O(n)$ preprocessing pass — essential when many queries hit the same static array.
- In-place techniques (swap-from-ends, read/write pointers) achieve $O(1)$ extra space instead of allocating a second array.
- Frequency counting beats sorting for anagram-style comparisons — $O(n)$ vs. $O(n \log n)$.
- Always state time and space complexity explicitly after presenting a solution — it's an expected part of the answer, not an optional add-on.

Learn More

GeeksforGeeks — Two Pointer Technique

<https://www.geeksforgeeks.org/dsa/two-pointers-technique/>

GeeksforGeeks — Window Sliding Technique

<https://www.geeksforgeeks.org/dsa/window-sliding-technique/>

GeeksforGeeks — Prefix Sum Array

<https://www.geeksforgeeks.org/dsa/prefix-sum-array-implementation-applications-competitive-programming/>